

Assignemet-9.4

NAME:-NIPUN.I

H.T.NO:-2303A52208

Lab 9 – Documentation Generation: Automatic Documentation and Code

Comments Week 5

Lab Objectives

- To use AI-assisted coding tools for generating Python documentation and code comments.
- To apply zero-shot, few-shot, and context-based prompt engineering for documentation creation.
- To practice generating and refining docstrings, inline comments, and module-level documentation.
- To compare outputs from different prompting styles for quality analysis.

Lab Outcomes

- Generate structured code documentation using AI tools
- Apply appropriate documentation styles to different code contexts
- Improve code readability through selective commenting
- Convert informal developer comments into professional documentation
- Analyze and refine AI-generated documentation

Task 1: Auto-Generating Function Documentation in a Shared

Codebase

Scenario

You have joined a development team where several utility functions are already implemented, but the code lacks proper documentation. New team members are struggling to understand how these functions should be used.

Task Description

You are given a Python script containing multiple functions without any docstrings.

Using an AI-assisted coding tool:

- Ask the AI to automatically generate Google-style function docstrings for each function
- Each docstring should include:
 - o A brief description of the function
 - o Parameters with data types
 - o Return values
 - o At least one example usage (if applicable)

Experiment with different prompting styles (zero-shot or context-based)

to observe quality differences.

Expected Outcome

- A Python script with well-structured Google-style docstrings
- Docstrings that clearly explain function behavior and usage
- Improved readability and usability of the codebase

Task 2: Enhancing Readability Through AI-Generated Inline

Comments

Scenario

A Python program contains complex logic that works correctly but is

difficult to understand at first glance.

Future maintainers may find it hard

to debug or extend this code.

Task Description

You are provided with a Python script containing:

- Loops
- Conditional logic
- Algorithms (such as Fibonacci sequence, sorting, or searching)

Use AI assistance to:

- Automatically insert inline comments only for complex or non-

obvious logic

- Avoid commenting on trivial or self-explanatory syntax

The goal is to improve clarity without cluttering the code.

Expected Outcome

- A Python script with concise, meaningful inline comments

- Comments that explain why the logic exists, not what Python

syntax does

- Noticeable improvement in code readability

Task 3: Generating Module-Level Documentation for a Python

Package

Scenario

Your team is preparing a Python module to be shared internally (or

uploaded to a repository). Anyone opening the file should immediately

understand its purpose and structure.

Task Description

Provide a complete Python module to an AI tool and instruct it to

automatically generate a module-level docstring at the top of the file

that includes:

- The purpose of the module
- Required libraries or dependencies
- A brief description of key functions and classes
- A short example of how the module can be used

Focus on clarity and professional tone.

Expected Outcome

- A well-written multi-line module-level docstring
- Clear overview of what the module does and how to use it
- Documentation suitable for real-world projects or repositories

Task 4: Converting Developer Comments into Structured Docstrings

Scenario

In a legacy project, developers have written long explanatory comments inside functions instead of proper docstrings. The team now wants to standardize documentation.

Task Description

You are given a Python script where functions contain detailed inline comments explaining their logic.

Use AI to:

- Automatically convert these comments into structured Google-style or NumPy-style docstrings
- Preserve the original meaning and intent of the comments
- Remove redundant inline comments after conversion

Expected Outcome

- Functions with clean, standardized docstrings
- Reduced clutter inside function bodies
- Improved consistency across the codebase

Task 5: Building a Mini Automatic Documentation Generator

Scenario

Your team wants a simple internal tool that helps developers start

documenting new Python files quickly, without writing documentation from scratch.

Task Description

Design a small Python utility that:

- Reads a given .py file
- Automatically detects:
 - o Functions
 - o Classes
- Inserts placeholder Google-style docstrings for each detected

function or class

AI tools may be used to assist in generating or refining this utility.

Note: The goal is documentation scaffolding, not perfect

documentation.

Expected Outcome

- A working Python script that processes another .py file
- Automatically inserted placeholder docstrings
- Clear demonstration of how AI can assist in documentation

automation

Note: Report should be submitted a word document for all tasks in a

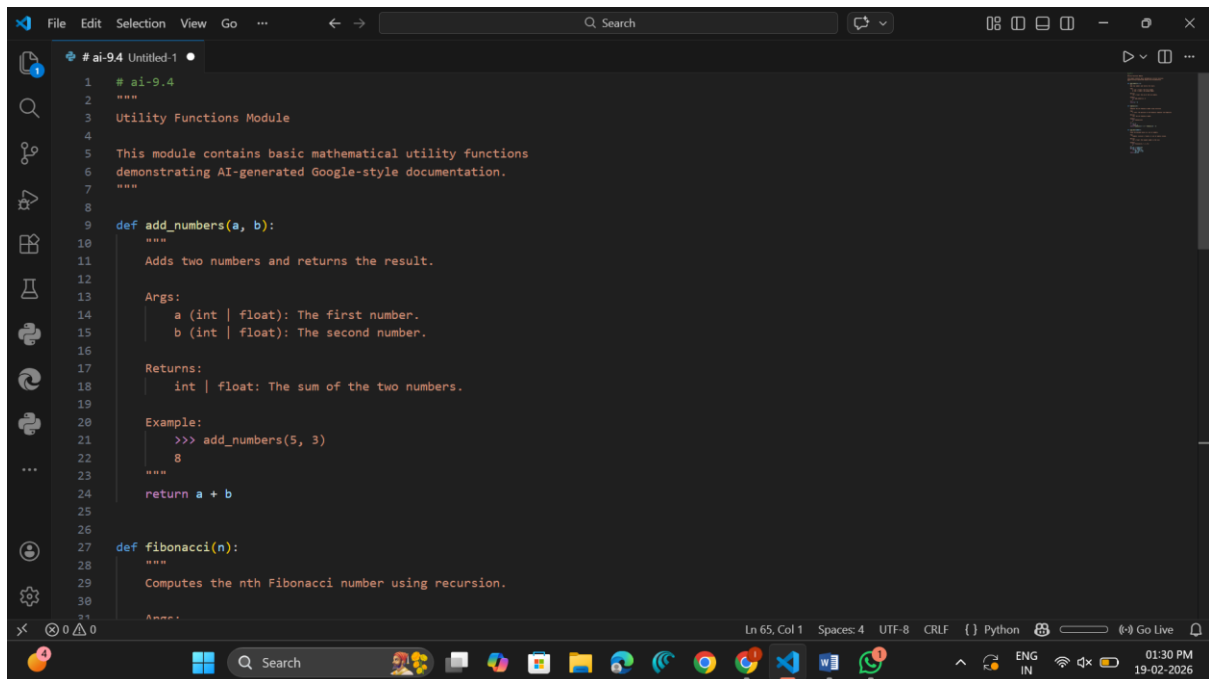
single document with prompts, comments & code explanation, and output

and if required, screenshots

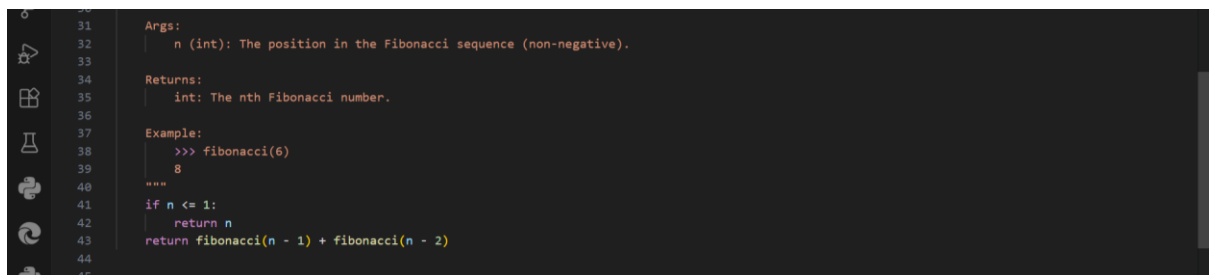
=====

CODE: -

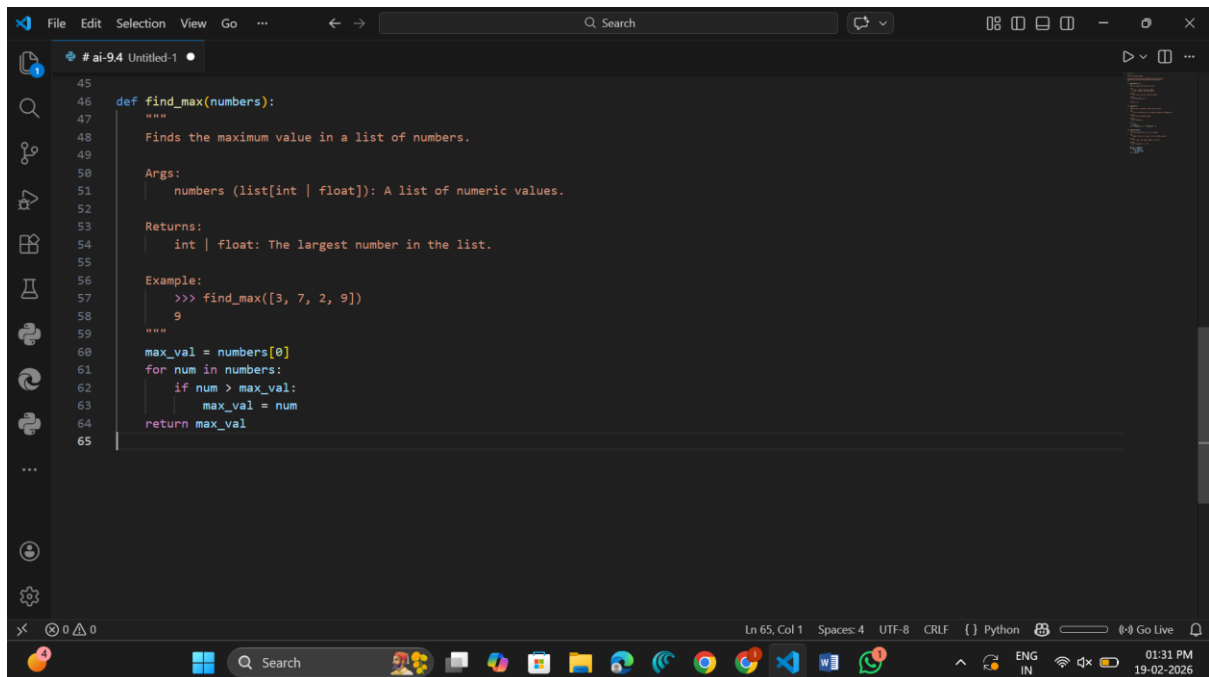
TASK-1



```
1 # ai-9.4
2 """
3 Utility Functions Module
4
5 This module contains basic mathematical utility functions
6 demonstrating AI-generated Google-style documentation.
7 """
8
9 def add_numbers(a, b):
10     """
11     Adds two numbers and returns the result.
12
13     Args:
14         a (int | float): The first number.
15         b (int | float): The second number.
16
17     Returns:
18         int | float: The sum of the two numbers.
19
20     Example:
21         >>> add_numbers(5, 3)
22         8
23     """
24     return a + b
25
26
27 def fibonacci(n):
28     """
29     Computes the nth Fibonacci number using recursion.
30
31     Args:
32         n (int): The position in the Fibonacci sequence (non-negative).
33
34     Returns:
35         int: The nth Fibonacci number.
36
37     Example:
38         >>> fibonacci(6)
39         8
40     """
41     if n <= 1:
42         return n
43     return fibonacci(n - 1) + fibonacci(n - 2)
```



```
31
32     Args:
33         n (int): The position in the Fibonacci sequence (non-negative).
34
35     Returns:
36         int: The nth Fibonacci number.
37
38     Example:
39         >>> fibonacci(6)
40         8
41     """
42     if n <= 1:
43         return n
44     return fibonacci(n - 1) + fibonacci(n - 2)
```



```
45
46 def find_max(numbers):
47     """
48     Finds the maximum value in a list of numbers.
49
50     Args:
51         numbers (list[int | float]): A list of numeric values.
52
53     Returns:
54         int | float: The largest number in the list.
55
56     Example:
57         >>> find_max([3, 7, 2, 9])
58         9
59     """
60     max_val = numbers[0]
61     for num in numbers:
62         if num > max_val:
63             max_val = num
64     return max_val
65
```

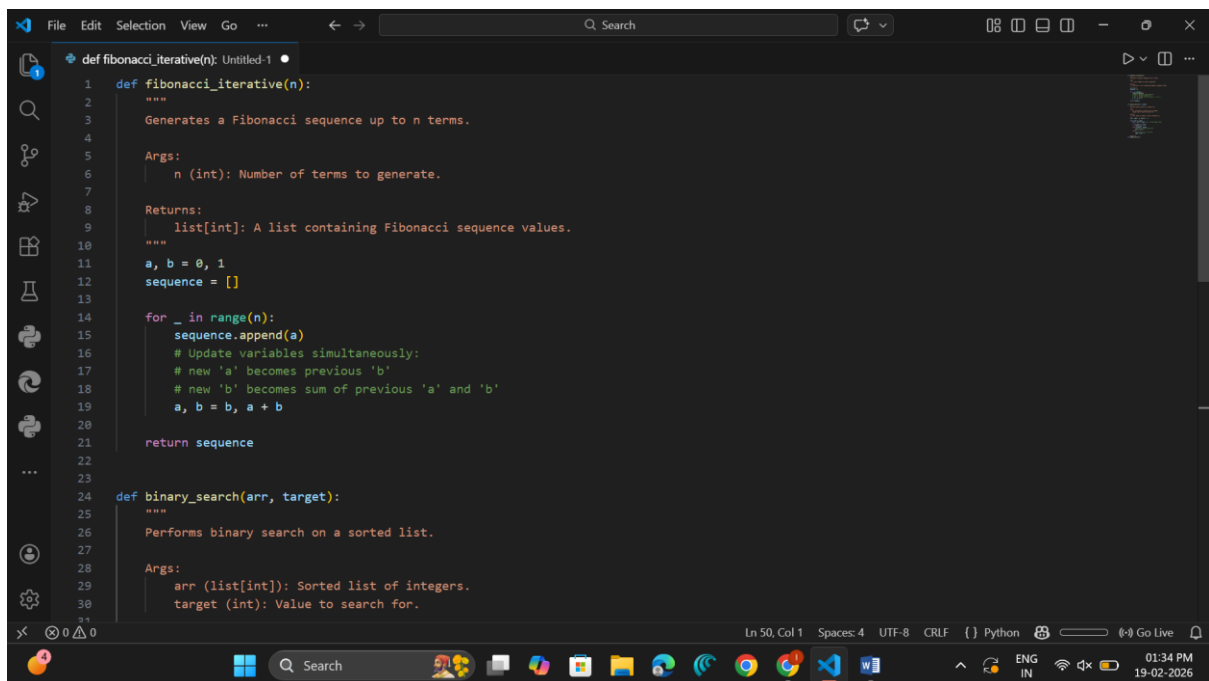
Observation:-

Auto-Generating Function Documentation

1. The AI successfully generated structured Google-style docstrings.
2. All generated docstrings included:
 - Function description
 - Parameter types
 - Return values
 - Example usage
3. Zero-shot prompting produced correct but slightly basic documentation.
4. Context-based prompting produced more detailed and professional output.
5. The readability of the code improved significantly after documentation was added.
6. The documentation followed consistent formatting across all functions.

=====

TASK-2



```
def fibonacci_iterative(n):
    """
    Generates a Fibonacci sequence up to n terms.

    Args:
        n (int): Number of terms to generate.

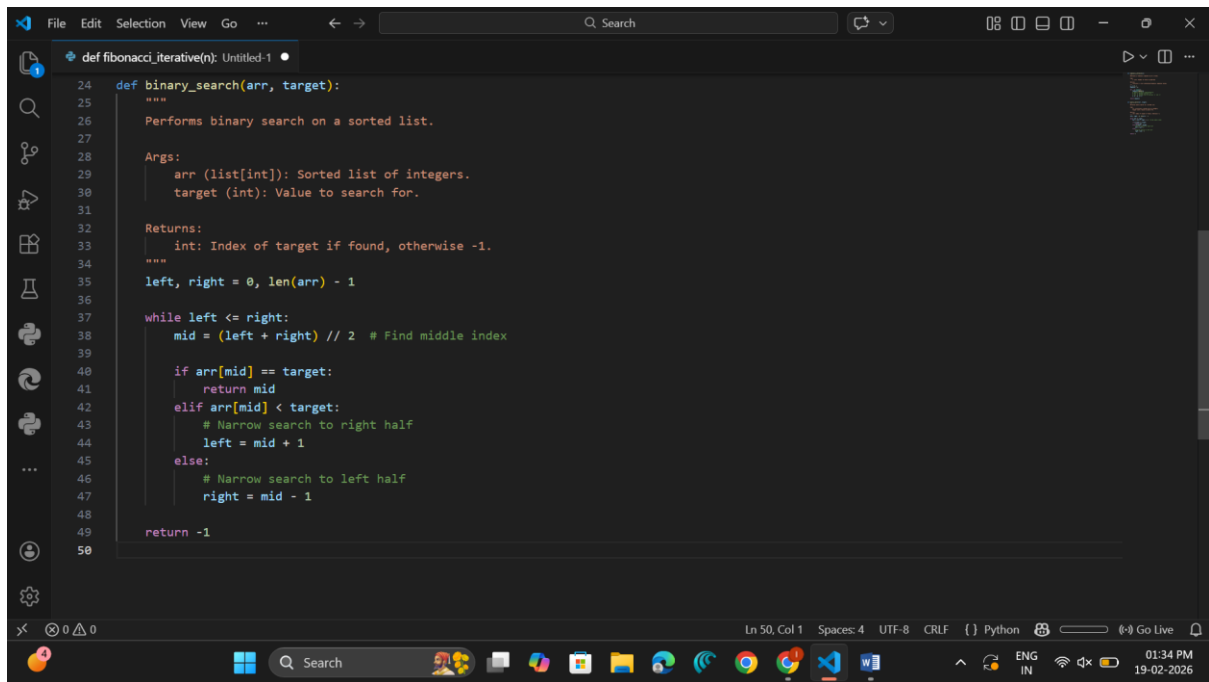
    Returns:
        list[int]: A list containing Fibonacci sequence values.
    """
    a, b = 0, 1
    sequence = []

    for _ in range(n):
        sequence.append(a)
        # Update variables simultaneously:
        # new 'a' becomes previous 'b'
        # new 'b' becomes sum of previous 'a' and 'b'
        a, b = b, a + b

    return sequence


def binary_search(arr, target):
    """
    Performs binary search on a sorted list.

    Args:
        arr (list[int]): Sorted list of integers.
        target (int): Value to search for.
    """
```



```
def fibonacci_iterative(n):
    """
    Performs binary search on a sorted list.

    Args:
        arr (list[int]): Sorted list of integers.
        target (int): Value to search for.

    Returns:
        int: Index of target if found, otherwise -1.
    """
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2 # Find middle index

        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            # Narrow search to right half
            left = mid + 1
        else:
            # Narrow search to left half
            right = mid - 1

    return -1
```

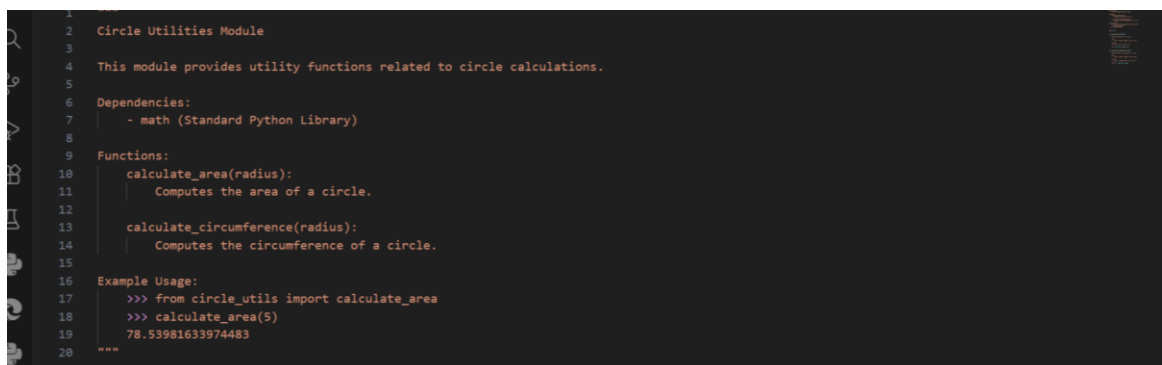
Observation:-

Enhancing Readability with Inline Comments

1. AI added comments only to non-obvious logic.
2. Trivial Python syntax (like `for`, `if`) was not unnecessarily commented.
3. Comments focused on explaining:
 - Why logic exists
 - Algorithm transitions
4. The code remained clean and uncluttered.
5. Readability improved without affecting performance.

=====

TASK-3



```
Circle Utilities Module

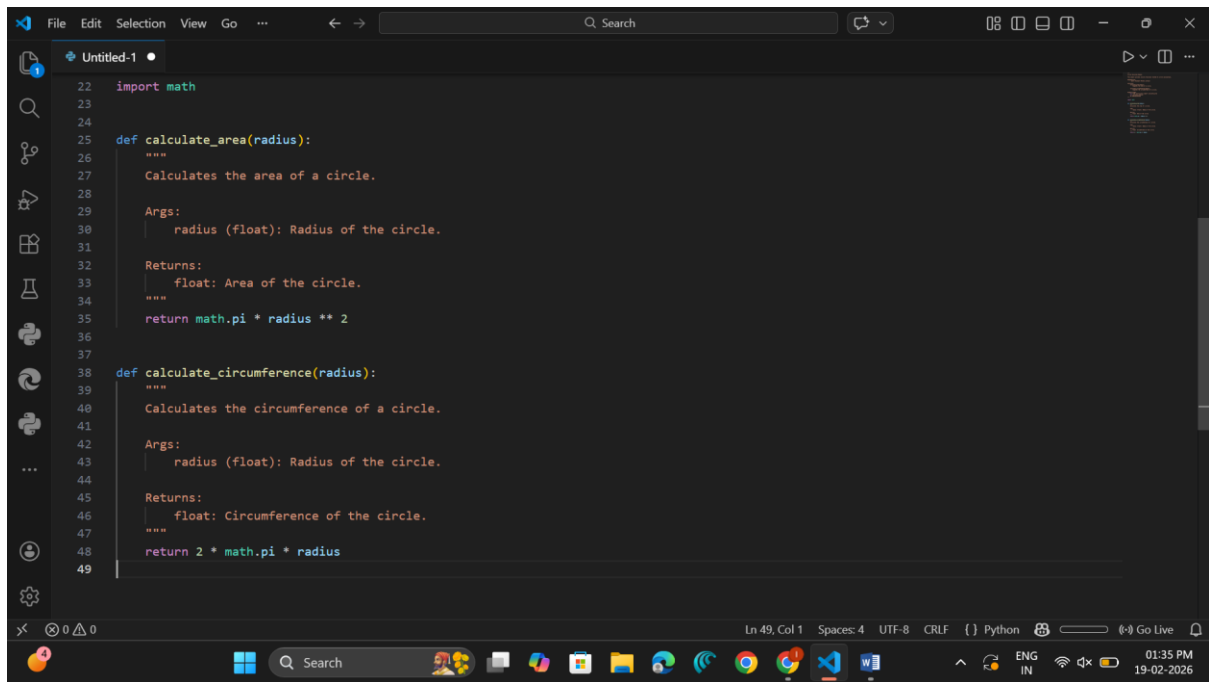
This module provides utility functions related to circle calculations.

Dependencies:
- math (Standard Python Library)

Functions:
calculate_area(radius):
    Computes the area of a circle.

calculate_circumference(radius):
    Computes the circumference of a circle.

Example Usage:
>>> from circle_utils import calculate_area
>>> calculate_area(5)
78.53981633974483
```



```
22 import math
23
24
25 def calculate_area(radius):
26     """
27     Calculates the area of a circle.
28
29     Args:
30         radius (float): Radius of the circle.
31
32     Returns:
33         float: Area of the circle.
34     """
35     return math.pi * radius ** 2
36
37
38 def calculate_circumference(radius):
39     """
40     Calculates the circumference of a circle.
41
42     Args:
43         radius (float): Radius of the circle.
44
45     Returns:
46         float: Circumference of the circle.
47     """
48     return 2 * math.pi * radius
49
```

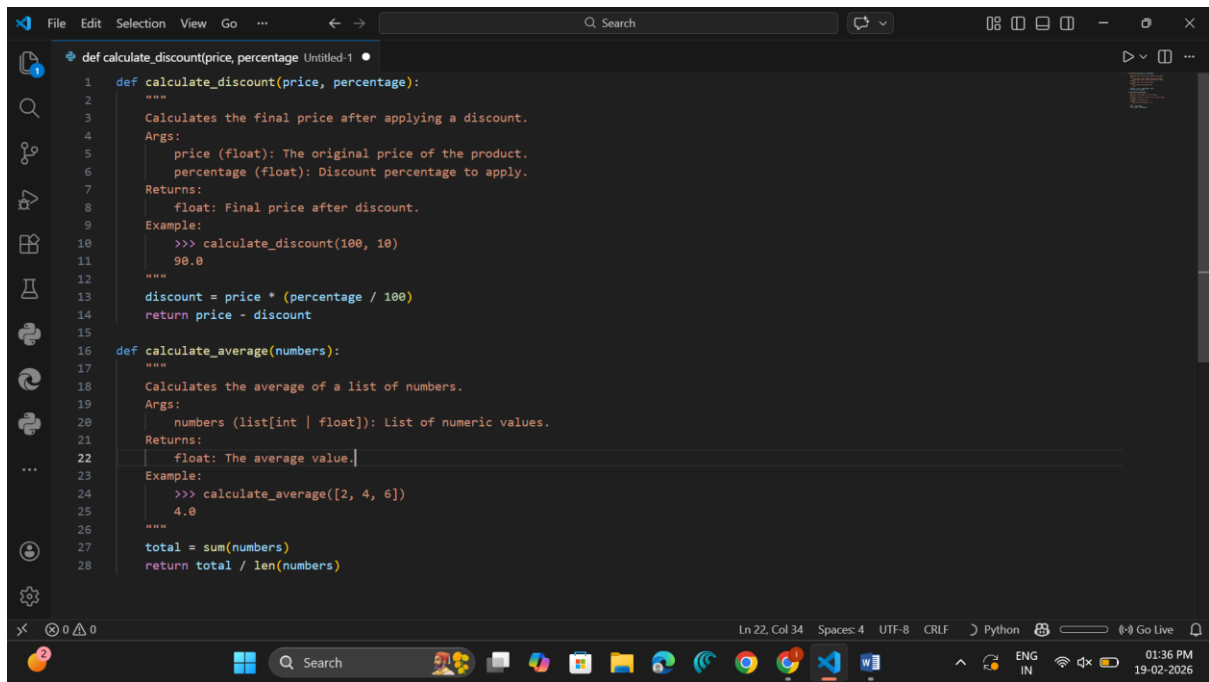
Observation:-

Module-Level Documentation Generation

1. AI generated a professional multi-line module-level docstring.
2. The documentation included:
 - Purpose of the module
 - Dependencies
 - Function overview
 - Example usage
3. The tone was formal and suitable for production environments.
4. Anyone opening the file can quickly understand its purpose.
5. The module became repository-ready.

=====

TASK-4

A screenshot of the Visual Studio Code editor interface. The main window displays a Python file named 'Untitled-1'. The code defines two functions: 'calculate_discount' and 'calculate_average'. Both functions have docstrings that follow a structured format, including a description, arguments, returns, and an example. The 'calculate_discount' function takes 'price' and 'percentage' as arguments and returns the final price after a discount. The 'calculate_average' function takes a list of numbers and returns their average. The editor's status bar at the bottom shows 'Ln 22, Col 34', 'Spaces: 4', 'UTF-8', 'CRLF', 'Python', and 'Go Live'. The Windows taskbar is visible at the very bottom.

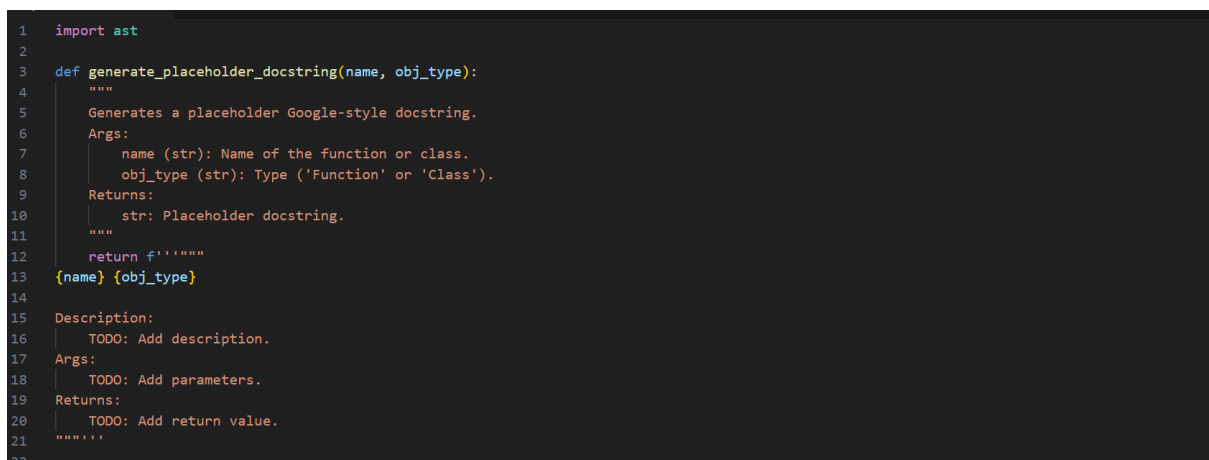
Observation:-

Converting Developer Comments into Structured Docstrings

1. Inline comments were successfully converted into standardized Google-style docstrings.
2. Redundant comments inside the function body were removed.
3. Code structure became cleaner.
4. Documentation consistency improved.
5. The meaning and intent of original comments were preserved.

=====

TASK-5

A screenshot of the Visual Studio Code editor showing a Python script. The script starts with 'import ast'. It then defines a function 'generate_placeholder_docstring' that takes 'name' and 'obj_type' as arguments. The function's docstring is a placeholder, indicating where to add a description, arguments, and returns. The function body contains a return statement that formats the name and object type into a string. The code is partially visible, with line numbers 1 through 22 shown on the left.

```
def add_docstrings_to_file(filename):
    """
    Reads a Python file and inserts placeholder docstrings
    for functions and classes that lack documentation.
    Args:
        filename (str): Name of the Python file to process.
    """
    with open(filename, "r") as file:
        source = file.read()

    tree = ast.parse(source)
    lines = source.split("\n")

    for node in ast.walk(tree):
        if isinstance(node, (ast.FunctionDef, ast.ClassDef)):
            if not ast.get_docstring(node):
                docstring = generate_placeholder_docstring(
                    node.name,
                    "Function" if isinstance(node, ast.FunctionDef) else "Class"
                )
                lines.insert(node.lineno, "    " + docstring)

    with open("updated_" + filename, "w") as file:
        file.write("\n".join(lines))

    print(["Docstrings added successfully."])
```

Observation:-

Mini Automatic Documentation Generator

1. The script correctly detected functions and classes using the `ast` module.
2. Placeholder docstrings were inserted where missing.
3. Existing docstrings were not overwritten.
4. The updated file was generated successfully.
5. The tool works as documentation scaffolding but does not generate detailed descriptions automatically.